Sugerencias de componentes y conexionado — Propuesta 2 de circuito CanSat

Para: compañero/a de IEM (revisión de hardware)
De: Florencia González (ISI 1)
Fecha: 2026-11-10
Estado: para analizar y responder
Circuito de referencia: `Propuesta 2 de circuito cansat.jpeg`
Documento relacionado: `docs/02-arquitectura/propuesta-circuito-cansat.pdf`

---

1. Para qué sirve este documento

La Propuesta 2 es el circuito que vamos a usar. El firmware del OBC (ESP32-S3) se está escribiendo para esos componentes reales, con sus direcciones I2C y su cableado.

Este documento **no reemplaza el esquema eléctrico**. Es una lista de verificación para que IEM revise, antes de soldar y antes de que el firmware quede cerrado:

1. Qué decisiones hay que tomar y cuál conviene.
2. Qué riesgos eléctricos conviene evitar.
3. Qué espera el firmware de cada bloque.
4. Qué queda pendiente de definir (radio, payload, pines definitivos).

La simulación en Wokwi se hará **después**, en un archivo aparte, solo como material didáctico. No define el hardware.

---

2. Resumen ejecutivo

- El mapa de direcciones I2C de la Propuesta 2 **no tiene colisiones** en el bus real.
- La única colisión aparece en la simulación: el DS1307 de Wokwi usa `0x68`, igual que el MPU-6050. Por eso la simulación usa un bus auxiliar. En la placa real el RTC es un **RV-3028-C7 en** `0x52` y va en el bus principal.
- El riesgo eléctrico más importante es **alimentar el servo desde el regulador del ESP32-S3**. Un SG90 puede pedir ~700 mA de pico y provocar un reinicio justo en el apogeo.
- El segundo riesgo es **alimentar los TSL251RD a 5 V**: su salida analógica puede superar los 3.3 V y dañar el ADS1115.
- El tercer riesgo es **ubicar el LIS3MDL cerca de cables de corriente o del servo**: el campo magnético falsea la medición.
- Faltan definir tres cosas que afectan el firmware: **rol del XBee**, **protocolo con la Raspberry Pi Zero 2W / XIAO** y **pinout definitivo**.

---

3. Decisiones que necesito que IEM valide

#	Tema	Supuesto actual	Recomendación	Qué pasa si no se define
1	Alimentación del servo	Desde el regulador del ESP32	Riel de 5 V aparte, masa común y capacitor de 470–1000 µF	Reinicios por caída de tensión en el apogeo
2	Shunt del INA219	0.1 Ω	Confirmar el valor del módulo	La corriente medida queda mal escalada
3	Batería	2S Li-ion 21700 con BMS	Confirmar 1S o 2S	Cambia la etapa de potencia (buck o boost)

#	Tema	Supuesto actual	Recomendación	Qué pasa si no se define
4	Variante del ESP32-S3	DevKitC-1 44 pines	Confirmar N8, N8R8 o N16R8	Si es N8R8, GPIO 33–37 no están disponibles
5	TSL251RD	Salida analógica de tensión	Confirmar el sufijo exacto del componente	Si fuera salida en frecuencia, cambia todo el front-end
6	XBee	Sin implementar	Definir modelo y rol (suelo o payload)	No se puede escribir el enlace de radio
7	Raspberry Pi Zero 2W / XIAO	Sin protocolo	Definir quién es maestro, baudios y trama	No se puede escribir el enlace con el payload
8	Pinout definitivo	Borrador	Cerrar pines con IEM 3	El firmware queda con pines provisorios

---

4. Arquitectura de alimentación recomendada



Reglas:

- **Masa común** entre todos los rieles. Sin masa común, el servo y el ADS1115 se comportan de forma errática.
- **Capacitor de 470–1000 µF** en el riel de 5 V, lo más cerca posible del servo.
- **Capacitor de 100 nF + 10 µF** en la entrada de cada sensor I2C.
- No alimentar el pin `3V3` del DevKitC-1 y el USB al mismo tiempo.
- Si la Raspberry Pi comparte el riel de 5 V con el servo, poner un buck separado o un filtro LC. La RPi puede pedir picos de ~500 mA y el servo ~700 mA; juntos superan un buck chico.

---

5. Bus I2C

5.1 Mapa de direcciones

Componente	Dirección	Bus	Nota
MPU-6050	`0x68`	Principal	Acelerómetro + giroscopio
BMP280	`0x76` o `0x77`	Principal	Autodetección; el módulo suele venir en `0x76`
LIS3MDL	`0x1C`	Principal	Magnetómetro
INA219	`0x40`	Principal	Tensión y corriente
ADS1115	`0x48`	Principal	ADC de 4 canales

Componente	Dirección	Bus	Nota
FRAM MB85RC256V	`0x50`	Principal	Log de vuelo
RV-3028-C7	`0x52`	Principal	RTC real
DS1307 (solo simulación)	`0x68`	Auxiliar	Exclusivo de Wokwi; colisiona con el MPU

Conclusión: en la placa real no hay conflicto de direcciones. El bus auxiliar (GPIO 41/42) es solo para la simulación.

5.2 Pull-ups

- Dejar **un solo par de pull-ups a 3.3 V** en todo el bus, de **2.2 kΩ a 4.7 kΩ**.
- Muchos módulos traen sus propias resistencias. Si se suman seis módulos, la resistencia equivalente queda demasiado baja y el bus consume corriente de más.
- Revisar cada módulo y, si hace falta, levantar o cortar los pull-ups duplicados.
- A 400 kHz y con cables cortos, 2.2 kΩ funciona bien. Si el cable al ADS1115 o al RTC es largo, bajar a 100 kHz.

5.3 Orden físico del bus

Conviene que el bus recorra los sensores en este orden, para que el RTC y la FRAM queden cerca del ESP32:

```
ESP32-S3 → MPU-6050 → BMP280 → LIS3MDL → INA219 → ADS1115 → FRAM → RTC
```

No es obligatorio, pero acorta el cable y reduce capacidad parásita.

---

6. Recomendaciones por componente

6.1 ESP32-S3 DevKitC-1 (OBC)

- Confirmar variante: **N8** (sin PSRAM), **N8R8** o **N16R8**.
- Si es **N8R8** (PSRAM octal), **GPIO 33–37 no se pueden usar**.
- Evitar para señales críticas: GPIO 0, 3, 45 y 46 (strapping), GPIO 19/20 (USB).
- Alimentar por `5V` / `VIN` desde el buck de 5 V, o por USB durante desarrollo. No ambas cosas a la vez.
- Dejar test points de `3V3`, `GND`, `TX`, `RX` y del bus I2C para poder medir con osciloscopio.

6.2 MPU-6050 (acelerómetro + giroscopio)

- Dirección `0x68`.
- Si se usa el módulo GY-521, admite 3.3 V o 5 V porque trae regulador y level shifter. Si se usa el chip pelado, **solo 3.3 V**.
- Montaje **rígido** al chasis, con los ejes alineados al cuerpo del CanSat.
- Evitar montarlo sobre espuma blanda: agrega ruido y retardo.
- El firmware lo muestrea a 100 Hz.

6.3 BMP280 (presión y temperatura)

- Dirección `0x76` o `0x77`; el firmware autodetecta.
- Alimentación 3.3 V.
- Va en un compartimento **ventilado**, lejos del regulador y del servo.
- No sellarlo con silicona ni pegamento: necesita intercambiar presión con el exterior.
- Es el sensor principal para la detección de apogeo (caída de 5 m sobre los 100 m).

6.4 LIS3MDL (magnetómetro)

- Dirección `0x1C`, alimentación 3.3 V.

- **Ubicación crítica:** lejos del shunt del INA219, de los cables de batería y del servo. La corriente genera campo magnético y falsea la medición.
- No usar el magnetómetro para decisiones de vuelo. Solo como dato de telemetría.
- El firmware lee `WHO_AM_I` (`0x0F`) y los 6 bytes de salida.

6.5 INA219 (tensión y corriente)

- Dirección `0x40`.
- Se conecta en **alta side**: entre el positivo de la batería y el resto del circuito.
- Shunt asumido: **0.1 Ω**. Con ese valor mide bien hasta ~2 A y resuelve ~0.1 mA.
- Si la corriente de pico supera los 2 A, conviene un shunt de **0.05 Ω** y recalibrar el firmware.
- Tensión máxima de bus: 32 V. Una batería 2S (8.4 V) queda cómoda.
- Verificar en el módulo si el shunt es de 0.1 Ω o de otro valor; el firmware tiene ese número como constante.

6.6 ADS1115 (ADC de 4 canales)

- Dirección `0x48`, alimentación 3.3 V.
- Se usa para los cuatro canales ópticos.
- Configurar en modo **single-ended** y rango de entrada **±4.096 V** (o el que cubra los 0–3.3 V de los sensores).
- Agregar **R de serie de 1–10 kΩ + capacitor a masa** en cada entrada: filtran ruido del servo y del motor.
- No conectar entradas por encima de `VDD + 0.3 V`.

6.7 TSL251RD (sensores ópticos)

- Son *light-to-voltage*: salida **analógica** proporcional a la luz.
- **Alimentarlos a 3.3 V**, no a 5 V. Si se alimentan a 5 V, la salida puede superar los 3.3 V y dañar el ADS1115.
- Confirmar el sufijo exacto del componente. Si fuera una variante con salida en frecuencia, el front-end cambia por completo.
- Calibrar en banco: luz ambiente, luz tapada y luz directa, para definir los umbrales.
- Usar el ADS1115, no el ADC interno del ESP32-S3. Los GPIO 1/2/3/6 son **solo de simulación**.

6.8 RV-3028-C7 (RTC real)

- Dirección `0x52`, alimentación 3.3 V.
- Necesita cristal de **32.768 kHz** y, si se quiere mantener la hora entre vuelos, batería o supercapacitor de respaldo.
- Tiene EEPROM y RAM internas; el firmware solo usa hora y fecha.
- Va en el **bus principal**, no en el auxiliar. El bus auxiliar es exclusivo de la simulación con DS1307.
- Revisar el mapa de registros en `Datasheets/RV-3028-C7.pdf` antes de soldar.

6.9 FRAM MB85RC256V (memoria de vuelo)

- Dirección `0x50`, alimentación 3.3 V.
- 32 KB, I2C, sin desgaste de escritura. Ideal para log continuo.
- Conectar el pin **WP** a masa, o a un GPIO si se quiere proteger la memoria durante el vuelo.
- No reemplazar por NVS: NVS es para configuración, no para log de alta frecuencia.
- El firmware guarda cada muestra con número de secuencia y CRC.

6.10 GPS NEO-M8N

- Alimentación 3.3 V, UART a **9600 baudios**.
- Cruzar TX/RX: TX del GPS → RX del ESP32, TX del ESP32 → RX del GPS.
- Antena con plano de masa, lejos del XBee y del servo.
- Si se quiere *hot start*, necesita batería de respaldo.
- El firmware parsea NMEA y extrae latitud, longitud, altitud, satélites y estado del fix.

6.11 Servo SG90

- Rango de alimentación 4.8–6 V. **No alimentar desde el regulador del ESP32-S3.**
- Corriente de pico ~700 mA; el arranque puede ser mayor.
- Riel de 5 V aparte, masa común, capacitor de 470–1000 µF.
- La señal PWM de 3.3 V del ESP32 suele ser suficiente para el SG90.
- Si el servo mueve una carga grande, agregar un MOSFET o un driver y alimentar la carga aparte.

6.12 XBee (radio)

- **Pendiente de definir.** No se implementa el enlace hasta saber:
- modelo exacto (XBee 3, Pro 900HP, etc.);
- si es enlace al suelo o al payload;
- UART, baudios y antena.
- El XBee trabaja a 3.3 V en UART. No necesita level shifter si el ESP32-S3 también es 3.3 V.
- Mantener la antena lejos del GPS y del magnetómetro.

6.13 Raspberry Pi Zero 2W / XIAO ESP32-S3 Sense (payload)

- **Pendiente de definir.** No se implementa el protocolo hasta saber:
- quién es maestro (OBC o payload);
- UART, SPI o USB;
- baudios y formato de trama.
- La RPi Zero 2W pide 5 V y puede consumir ~500 mA de pico. Riel separado o filtro LC.
- El XIAO puede ir a 3.3 V o 5 V; confirmar con el datasheet.

6.14 Batería y protección

- Confirmar **1S o 2S**. La suposición actual es **2S Li-ion 21700 con BMS**.
- Fusible o eFuse en la entrada.
- Diodo de protección de polaridad inversa.
- TVS en las líneas que salen del gabinete (GPS, radio, servo).
- Conectores con traba (JST-GH o bornera con tornillo) y alivio de tensión. En vuelo, un Dupont se sale.

---

7. Qué espera el firmware de cada bloque

Bloque	Qué necesita el firmware	Si falla
MPU-6050	Lectura de aceleración y giro a 100 Hz	Reporta `IMU:FAIL`, sigue con barómetro
BMP280	Presión y temperatura a 10 Hz	Reporta `BARO:FAIL`, no detecta apogeo por presión
LIS3MDL	`WHO_AM_I` y 6 bytes de salida	Reporta `MAG:FAIL`, campos en cero
INA219	Tensión de bus y corriente	Reporta `PWR:FAIL`, no inventa valores
ADS1115	4 canales single-ended	Reporta `OPT:FAIL`, canales en cero
FRAM	Escritura/lectura con secuencia y CRC	Reporta `FRAM:FAIL`, log en RAM con advertencia
RV-3028-C7	Hora y fecha	Reporta `RTC:FAIL`, usa `millis()`
GPS	NMEA por UART	Reporta `GPS:NOFIX`, sigue el vuelo
Servo	PWM de liberación	Si no responde, el mecanismo queda trabado

Regla de diseño: en vuelo, el firmware **nunca rellena un sensor faltante con un valor inventado**. Si un sensor no responde, lo reporta y sigue. Los datos sintéticos existen solo en el modo simulación (`CANSAT_SIMULATION`).

---

8. Pinout borrador (a confirmar con IEM 3)

Función	GPIO	Estado
I2C SDA	8	Borrador
I2C SCL	9	Borrador
I2C auxiliar SDA (solo simulación)	41	Solo Wokwi
I2C auxiliar SCL (solo simulación)	42	Solo Wokwi
GPS RX	18	Borrador
GPS TX	17	Borrador
Óptico 1 (solo simulación)	1	Solo Wokwi
Óptico 2 (solo simulación)	2	Solo Wokwi
Óptico 3 (solo simulación)	3	Solo Wokwi
Óptico 4 (solo simulación)	6	Solo Wokwi
Botón de armado	5	Borrador
LED LaunchPad (azul)	10	Borrador
LED Ascent (verde)	11	Borrador
LED Descent (amarillo)	12	Borrador
LED 2.º paracaídas (rojo)	13	Borrador
LED Landed (blanco)	14	Borrador
Servo	15	Borrador

Los pines 1/2/3/6 y 41/42 **no se usan en la placa real**: son exclusivos de la simulación. En la placa real, los ópticos van al ADS1115 y el RTC va al bus principal en `0x52`.

---

9. Checklist para IEM

Marcar lo que se va confirmando:

- ☐ Modelo exacto del ESP32-S3 (N8 / N8R8 / N16R8).
- ☐ Batería: 1S o 2S, capacidad y BMS.
- ☐ Riel de 5 V para servo, con capacitor y masa común.
- ☐ Shunt real del módulo INA219.
- ☐ Tipo de salida y sufijo exacto del TSL251RD.
- ☐ Alimentación de los TSL251RD a 3.3 V.
- ☐ Pull-ups del bus I2C (un solo par).
- ☐ Ubicación del LIS3MDL lejos de corrientes.
- ☐ Ventilación del BMP280.
- ☐ Cristal y respaldo del RV-3028-C7.
- ☐ Pin `WP` de la FRAM.
- ☐ Antena y respaldo del GPS.
- ☐ Modelo y rol del XBee.
- ☐ Protocolo con Raspberry Pi Zero 2W / XIAO.
- ☐ Pinout definitivo.
- ☐ Conectores con traba y alivio de tensión.
- ☐ Protección: fusible, polaridad inversa, TVS.

10. Preguntas abiertas

1. ¿El XBee es el enlace de telemetría al suelo o el enlace con el payload?
2. ¿La Raspberry Pi Zero 2W y el XIAO van a bordo? ¿Quién es maestro?
3. ¿El TSL251RD es la variante de salida analógica? ¿Cuántos canales van?
4. ¿La batería es 1S o 2S? ¿Hay BMS?
5. ¿La placa es N8, N8R8 o N16R8?
6. ¿Hay restricciones de espacio o de masa que obliguen a cambiar conectores o rieles?

11. Qué NO propongo

- No cambiar la Propuesta 2 ni reemplazar componentes por otros.
- No agregar módulos que no estén en el circuito sin justificarlo.
- No usar la simulación de Wokwi como contrato de hardware.
- No implementar radio ni payload hasta que se defina su rol.

12. Referencias

- `Propuesta 2 de circuito cansat.jpeg`
- `docs/02-arquitectura/propuesta-circuito-cansat.pdf`
- `Datasheets/ESP32-S3 - WROOM-1 WROOM-1U.pdf`
- `Datasheets/MPU-6000 MPU-6050.pdf`
- `Datasheets/BMP280.PDF`
- `Datasheets/LIS3MDL.PDF`
- `Datasheets/INA219.PDF`
- `Datasheets/ADS1113.PDF`
- `Datasheets/RV-3028-C7.pdf`
- `Datasheets/FRAM MB85RC256V.pdf`
- `Datasheets/NEO-M8.PDF`
- `Datasheets/XIAO ESP32-S3 Sense.pdf`
- `Datasheets/Raspberry Pi Zero 2W.pdf`